



PROFESSIONAL REFERENCE • VOLUMES 1 & 2

System Design Expert Reference

A practical field guide for architecture decisions, project planning, and 45-minute system design interviews.

28 system designs

Capacity estimation • Reliability • Data architecture • Distributed transactions

NAVIGATOR

Table of contents

Use pages 5–6 to frame a problem, pages 7–34 to select a design, and pages 35–38 to review cross-cutting decisions before implementation.

03	Chapter manifest	05	The 4-step framework
06	Estimation cheat sheet	07	Ch 1 — Scale from Zero to Millions of Users
08	Ch 2 — Back-of-the-Envelope Estimation	09	Ch 3 — System Design Framework
10	Ch 4 — Rate Limiter	11	Ch 5 — Consistent Hashing
12	Ch 6 — Key-Value Store	13	Ch 7 — Unique ID Generator
14	Ch 8 — URL Shortener	15	Ch 9 — Web Crawler
16	Ch 10 — Notification System	17	Ch 11 — News Feed System
18	Ch 12 — Chat System	19	Ch 13 — Search Autocomplete
20	Ch 14 — YouTube	21	Ch 15 — Google Drive
22	Ch 16 — Proximity Service	23	Ch 17 — Nearby Friends
24	Ch 18 — Google Maps	25	Ch 19 — Distributed Message Queue
26	Ch 20 — Metrics Monitoring and Alerting	27	Ch 21 — Ad Click Event Aggregation
28	Ch 22 — Hotel Reservation System	29	Ch 23 — Distributed Email Service
30	Ch 24 — S3-like Object Storage	31	Ch 25 — Real-time Gaming Leaderboard
32	Ch 26 — Payment System	33	Ch 27 — Digital Wallet
34	Ch 28 — Stock Exchange	35	Cross-cutting patterns
38	Pitfalls and wrap-up	39	Sources and scope

LIBRARY MAP

Chapter manifest 1/2

The installed library covers all 28 chapters. Folder names point to the local source set; diagram counts show where deeper visual material is available.

#	Chapter	Source directory	Diagrams
01	Scale from Zero to Millions of Users	01. Scaling/	12
02	Back-of-the-Envelope Estimation	02. Back Of the Envelope Estimation/	1
03	System Design Framework	03. System Design Framework/	0
04	Rate Limiter	04. Rate Limiter/	8
05	Consistent Hashing	05. Consistent Hashing/	10
06	Key-Value Store	06. Key-Value Store/	17
07	Unique ID Generator	07. Unique-Id Generator/	5
08	URL Shortener	08. URL Shortener/	7
09	Web Crawler	09. Web Crawler/	4
10	Notification System	10. Notification System/	6
11	News Feed System	11. News Feed System/	5
12	Chat System	12. Chat System/	16
13	Search Autocomplete	13. Search Autocomplete/	9
14	YouTube	14. Youtube/	14

LIBRARY MAP

Chapter manifest 2/2

Manifest continued: advanced location, streaming, observability, transaction, and finance designs.

#	Chapter	Source directory	Diagrams
15	Google Drive	15. Google Drive/	9
16	Proximity Service	16. Proximity Service/	15
17	Nearby Friends	17. Nearby Friends/	10
18	Google Maps	18. Google Maps/	22
19	Distributed Message Queue	19. Distributed Message Queue/	35
20	Metrics Monitoring and Alerting	20. Metrics Monitoring and Alerting System/	22
21	Ad Click Event Aggregation	21. Ad Click Event Aggregation/	29
22	Hotel Reservation System	22. Hotel Reservation System/	17
23	Distributed Email Service	23. Distributed Email Service/	14
24	S3-like Object Storage	24. S3-like Object Storage/	23
25	Real-time Gaming Leaderboard	25. Real-time Gaming Leaderboard/	23
26	Payment System	26. Payment System/	10
27	Digital Wallet	27. Digital Wallet/	25
28	Stock Exchange	28. Stock Exchange/	24

METHOD

The 4-step interview framework

Treat the interviewer as a teammate. Think aloud, make assumptions visible, and ask for buy-in before each deeper choice.

Treat the interviewer as a teammate. Think aloud; iterate on feedback.

Step	What	Time
1. Understand the problem & establish scope	Clarifying questions; pin functional/non-functional requirements	3–10 min
2. High-level design & get buy-in	Boxes + arrows; back-of-envelope math; walk key use cases	10–15 min
3. Deep dive	2–3 critical components the interviewer signals interest in	10–25 min
4. Wrap-up	Recap trade-offs, error handling, scaling 1M → 10M, follow-ups	3–5 min

Dos: clarify first; never go silent; spend effort on what defines the system (hash design for URL shorteners, latency + presence for chat, fanout for news feed). **Don'ts:** no premature full solutions; no over-engineering.

CAPACITY PLANNING

Back-of-the-envelope cheat sheet

Estimate the order of magnitude, not a fake precision. Write assumptions, label units, and separate average from peak.

Powers of 10: KB 10^3 , MB 10^6 , GB 10^9 , TB 10^{12} , PB 10^{15} . **Powers of 2:** $2^{10} \approx 1\text{K}$, $2^{20} \approx 1\text{M}$, $2^{30} \approx 1\text{G}$, $2^{40} \approx 1\text{T}$. **Quick conversions:** 1 day $\approx 86.4\text{k s} \approx 100\text{k s}$; **$\sim 2.5\text{M requests/day} \approx 30\text{ QPS}$** ; peak $\approx 2\text{--}5\times$ average.

Latency (every engineer should know): L1 0.5 ns · L2 7 ns · RAM 100 ns · compress 1 KB 3 μs · SSD random read 150 μs · HDD seek 10 ms · same-DC network 500 μs · inter-region RTT $\sim 100\text{--}150\text{ ms}$. *Moral:* memory is fast, disk seeks are slow; avoid seeks; compress before sending.

Availability: 99% $\approx 3.65\text{ days/yr}$ · 99.9% $\approx 8.8\text{ h}$ · 99.99% $\approx 53\text{ min}$ · 99.999% $\approx 5.3\text{ min}$. Cloud SLA $\geq 99.9\%$.

Rules: round aggressively; write assumptions; label units; quote QPS, peak QPS, storage, cache, servers.

Worked examples: Twitter — 300M MAU, 50% DAU, 2 tweets/day $\rightarrow \sim 3.5\text{k write QPS}$ ($\sim 7\text{k peak}$); 10% with 1 MB media $\rightarrow 30\text{ TB/day} \rightarrow \sim 55\text{ PB/5 yr}$. URL shortener — 100M/day $\approx 1,160\text{ writes/s}$, 10:1 read:write, 365B rows/10 yr $\approx 365\text{ TB}$; 7-char base-62 = $62^7 \approx 3.5\text{T IDs}$. Web crawler — 1B pages/month $\approx 400\text{/s}$ (peak 800), $\sim 30\text{ PB/5 yr}$. Email — 1B users, 10 sent/day $\rightarrow 100\text{k/s}$; $\sim 730\text{ PB/yr metadata}$. Stock exchange — 1B orders/day over 6.5 h $\rightarrow \sim 43\text{k avg QPS}$, $\sim 215\text{k peak}$.

DESIGN BLUEPRINT

Scale from Zero to Millions of Users

Ladder: single server → separate web/DB tiers → load balancer (stateless web tier) → master-slave DB replication (master writes, slaves read; read-heavy ⇒ more slaves) → cache tier → CDN → multi-DC (GeoDNS) → message queue for async → logging/metrics/automation → DB sharding. Each step fixes the previous

bottleneck. NoSQL when latency must be tiny, data is unstructured, you only serialize/deserialize, or volume is massive. Vertical vs horizontal scaling; cache-aside + LRU eviction; master failover ⇒ slave may be stale (recovery scripts); celebrity/hot-key ⇒ dedicated shard. Watch: cache ↔ DB write inconsistency; CDN invalidation.

01

DESIGN BLUEPRINT

Back-of-the-Envelope Estimation

Skill chapter — figures in the cheat sheet above. Habit: say assumptions aloud;

estimate QPS → peak QPS → storage → cache → servers. Process over precision.

02

DESIGN BLUEPRINT

System Design Framework

Process chapter — framework in §1 above.
Deep-dive targets = what defines the
system (hash design for URL shorteners,

latency/presence for chat, fanout for news
feed).

03

DESIGN BLUEPRINT

Rate Limiter

Server-side (client-side unreliable), standalone or as API-gateway/middleware; Redis counters; HTTP 429. **Token bucket** — fixed refill rate, bursty, simple, memory-efficient. *Default.* **Leaky bucket** — FIFO, fixed outflow; smooth output. **Fixed window** — simple but **edge bursts pass ~2× quota**. **Sliding-window log** —

accurate, memory-heavy. **Sliding-window counter** — blend; memory-efficient, approximate. *Practical Redis choice.* Distributed: Redis sorted sets + Lua for atomicity; centralized store; monitor to tune rules; multi-DC sync is eventually consistent.

04

DESIGN BLUEPRINT

Consistent Hashing

Naive `hash(key) % N` moves ~all keys on membership change. Ring $0 \dots 2^{160} - 1$ (SHA-1); nodes by IP/name; key owner = first node clockwise. Add/remove moves only the adjacent range. **Virtual nodes** $\Rightarrow$

balance (more vnodes = smaller std-dev), weighted for heterogeneous capacity; N replicas = N clockwise positions. Used by DynamoDB, Cassandra, Discord, Akamai, Maglev.

05

DESIGN BLUEPRINT

Key-Value Store

Distributed `put/get` for <10 KB pairs; HA, low latency, tunable consistency. Coordinator node proxies to the ring. Ring + N-way replication; quorum; **vector clocks** [server,version] (siblings = conflict $\Rightarrow$ client resolves; prune growth); gossip failure detection (≥ 2 sources before marking down); **sloppy quorum + hinted handoff** (temporary failures); **Merkle trees**

(permanent-failure sync; compare roots, descend mismatches, sync deltas); multi-DC replication. Write: commit log $\rightarrow$ memtable $\rightarrow$ SSTable. Read: cache $\rightarrow$ Bloom filter $\rightarrow$ SSTable. **CAP**: partitions unavoidable $\Rightarrow$ CA impossible; CP (block writes) vs AP (stale reads, sync later). **$W+R > N \Rightarrow$ strong consistency** ($N=3$, $W=R=2$ typical); $R=1, W=N$ favors reads; $W=1, R=N$ favors writes.

06

DESIGN BLUEPRINT

Unique ID Generator

Need: 64-bit, roughly-time-ordered, unique, >10k/sec. **Snowflake**: 1 sign + 41-bit ms timestamp (custom epoch) + 5-bit datacenter + 5-bit machine + 12-bit sequence (4,096/ms/machine, resets each ms). Decentralized, time-sortable; needs

NTP; tune widths per use case; handle same-ms overflow. Alternatives: multi-master auto-increment (not time-ordered, painful node changes), UUID (zero coordination, non-sortable), ticket server (simple SPOF).

07

DESIGN BLUEPRINT

URL Shortener

POST `api/v1/data/shorten {longUrl}` → shortURL; GET `api/v1/shortUrl` → 3xx redirect. 100M/day ≈ 1,160 writes/s; 10:1 read:write; 365 TB/10 yr. Web tier → cache → DB (replicated, sharded) → ID generator → rate limiter → analytics. Table: (id PK, shortURL, longURL). Shorten: return

existing if longURL seen, else generate ID → base-62 → store. Redirect: cache then DB. **Base-62 of ID:** collision-free but predictable (security). **Hash first 7 chars:** fixed length, collisions ⇒ retry with salt / Bloom check. **302 (not 301) when you need click analytics** — 301 is browser-cached, server never re-contacted.

08

DESIGN BLUEPRINT

Web Crawler

1B pages/month $\approx$ 400/s (peak 800). Seeds $\rightarrow$ **URL frontier** (FIFO) $\rightarrow$ downloader (DNS resolver + cache) $\rightarrow$ parser $\rightarrow$ content dedup (hash compare) $\rightarrow$ content storage (hot in memory, rest disk) $\rightarrow$ URL extractor $\rightarrow$ URL filter (blacklists) $\rightarrow$ URL dedup $\rightarrow$ URL storage. **Politeness:** per-host queues + workers, one request/host at a time, delay between tasks. **Priority:** front queues

(PageRank / update-frequency) + back queues (politeness), weighted selector; recrawl on update history. BFS over DFS (unbounded web depth); spider traps via URL length limits. Deep dive: robots.txt, consistent hashing for crawl servers, short timeouts, SSR (JS/AJAX), pluggable content modules.

09

DESIGN BLUEPRINT

Notification System

Soft-real-time multi-channel (APNS/FCM push, Twilio SMS, SendGrid email); 10M push + 1M SMS + 5M email/day; opt-out per channel. Triggers → notification servers (validation, metadata) → **queues (buffer + decouple)** → workers → providers. DB/cache **outside** the servers; horizontal

server scale; queue-depth monitoring to auto-scale workers. Reliability: persist + retry; dedup by event-ID; per-channel rate limits; templates; open/click tracking. AppKey/AppSecret auth. Deep dive: initial single-server SPOF; geographic delivery.

10

DESIGN BLUEPRINT

News Feed System

Reverse-chronological friends' posts; 10M DAU, $\leq 5,000$ friends/user. [POST/GET /v1/me/feed](#) . LB → web (auth, rate limit) → post service (DB + cache) → **fanout service** → feed cache → notifications. Cache post IDs only (not full objects); layers: feed, content, social graph, actions, counters. **Fanout on write (push)**: fast reads, costly

for celebrities. **Fanout on read (pull)**: cheap writes, slow reads. **Hybrid**: push for normal users, pull for celebrities. Pipeline: friend IDs from graph DB → filter muted/selective-share → queue (friend list, post ID) → workers append to feed caches with a cap. Deep dive: sharding + read replicas; cache hit-rate monitoring.

11

DESIGN BLUEPRINT

Chat System

Real-time 1:1 + small-group (≤ 100) chat, presence, multi-device, push; 50M DAU; permanent history. Stateless services + ZooKeeper service discovery (chat server by geo/capacity) + **stateful WebSocket chat servers** + presence + API + notification servers + KV history store. Sender: HTTP persistent; receiver: **WebSocket** (polling wasteful, long polling bad for inactive users). Group: copy to each recipient's inbox (simple, costly at scale); per-recipient

sync queue for pull. **Local per-channel sequence IDs suffice — global IDs are overkill.** Multi-device: `cur_max_message_id` per device. Presence: heartbeats; missed past threshold (~ 30) $\Rightarrow$ offline; per-friend-pair pub-sub works only for small groups. KV rationale: horizontal scale, low latency (Messenger/Discord precedent). Deep dive: delivery retry/queue; E2E encryption; media handling.

12

DESIGN BLUEPRINT

Search Autocomplete

Top-5 typeahead by popularity; 10M DAU, peak 48k QPS, <100 ms. **Data gathering:** analytics logs → aggregators (frequency tables) → weekly trie rebuild + snapshot → distributed trie cache / trie DB. **Query service:** trie lookup; find prefix node → traverse → sort → top-k; precomputed top-k per node; cap prefix ~50 chars. Shard by

prefix ranges (a-m, n-z; sub-shard for skew) with shard-map manager. Filter layer for deletions (hate speech), async physical delete. **Per-query real-time trie updates infeasible at billions/day** — batch weekly; trending via shorter windows/recency weighting. Deep dive: browser caching of frequent terms; country tries + Unicode.

13

DESIGN BLUEPRINT

YouTube

Upload + smooth adaptive streaming at low cost. Design: 5M DAU, 300 MB avg video, 1 GB max $\Rightarrow$ 150 TB/day. CDN math: $5M \times 5 \text{ videos} \times 0.3 \text{ GB} \times \$0.02 \approx$ **\$150k/day** — cost drives design. Clients $\rightarrow$ CDN (edge, DASH/HLS) + API servers $\rightarrow$ metadata DB + blob originals $\rightarrow$ **transcoding** (preprocessor / DAG scheduler / resource manager / workers) $\rightarrow$ transcoded storage $\rightarrow$ CDN. Transcoding: H.264/VP9; DAG parallelism (encode, thumbnails,

watermark); GOP-aligned chunk-parallel encoding with persisted intermediates for retry. Upload: blob + metadata in parallel; completion queue $\Rightarrow$ metadata update + notify. Speed: chunked resumable uploads, CDN upload hubs, queue-decoupled parallelism. Safety: pre-signed URLs, DRM, AES, watermarking. Cost: CDN only popular videos; on-demand encode for rare ones; ISP partnerships.

14

DESIGN BLUEPRINT

Google Drive

Upload/download, cross-device sync, sharing, revisions, notifications; 10M DAU; 10 GB/user; 500 KB avg upload $\Rightarrow$ ~500 PB. `files/upload?uploadType=resumable` (init $\rightarrow$ resumable URL $\rightarrow$ upload $\rightarrow$ monitor $\rightarrow$ resume); `files/download`; `files/list_revisions`. Block servers split files into ≤ 4 MB hashed blocks $\rightarrow$ S3 (+ cross-region) $\rightarrow$ Glacier cold $\rightarrow$ API servers (auth/metadata) $\rightarrow$ sharded metadata DB +

cache $\rightarrow$ notification service (long-polling pub-sub) $\rightarrow$ offline backup queue. **Delta sync** (modified blocks only); per-type compression; **block-hash dedup**; conflicts: first-writer wins, loser saved separately; versioning limits (recent prioritized); cold storage for inactive files. Deep dive: per-component failure handling (LB secondary, slave promotion, cross-region fetch, reconnect).

15

DESIGN BLUEPRINT

Proximity Service

Nearby-businesses search; 100M DAU, 200M businesses, ~5k search QPS. `GET`

`/v1/search/nearby?`

`latitude=&longitude=&radius=` (default 5,000 m); business CRUD. LB → stateless LBS (read-heavy, peaky) + business service → primary-replica DB; Redis (geohash → IDs; id → details); multi-region. Listing updates batch daily ⇒ replica lag acceptable. Naive 2-D scan fails; even grid fails on density. **Geohash**: one alphanumeric string, longer prefix = closer;

radius ⇒ precision (500 m → length 6).

Trap: boundary issue — nearby points can share no prefix; fix: query 8 neighbors.

Quadtree: in-memory 4-way split until ≤100 businesses/node; density-adaptive; k-nearest; harder updates; rebuild minutes ⇒ rolling deploys. **S2/Hilbert**: best for geofencing, hardest to build. **Cache keys = geohash, not raw GPS** (unstable as users move). Retrieval: radius → geohash length → self + 8 neighbors → parallel Redis lookups → rank by distance.

16

DESIGN BLUEPRINT

Nearby Friends

Real-time nearby-friends on constantly-moving locations (30 s refresh); the dynamic twin of proximity search. 1B users, 10% use it (100M/day), 10% concurrent $\Rightarrow$ ~334k updates/s; ~400 friends, 10% online $\Rightarrow$ **~14M fan-out pushes/s** peak; 5-mile radius; inactive after 10 min. LB $\rightarrow$ stateless REST servers (aux) + **stateful WebSocket servers (fanout)**; Redis `user_id  $\rightarrow$  (lat, long, timestamp)` with 10-min TTL; user DB (sharded by `user_id`); Cassandra location-history; **Redis pub/sub** — one

channel per user; WebSocket servers subscribe to friends' channels, forward after distance filtering. Scaling: ~200 GB channels but CPU-bound ~100k pushes/server $\Rightarrow$ ~140 Redis nodes; ZooKeeper channel mapping cached in-memory; consistent hashing limits churn; rescale off-peak (missing updates OK). Cap friend counts; whales absorbed by many servers; graceful LB drain on teardown. Variant: geohash-based pub/sub for "random nearby person".

17

DESIGN BLUEPRINT

Google Maps

Maps at 1B DAU: ingestion, navigation + ETA, rendering; traffic-aware, minimal client data/battery. ~200k avg QPS, 1M peak; ~70 PB tiles. **Location:** batched `POST`

`/v1/locations` → Cassandra (user_id partition, timestamp clustering; availability over consistency) → Kafka analytics.

Navigation: `GET /v1/nav?`

`origin&destination` ⇒

distance/duration/legs/polyline; geocoder

→ route planner → A-star/Dijkstra on

hierarchical routing tiles → **ML ETA** →

ranker. Rendering: **precomputed 256×256**

static tiles per zoom (quadruples per level) from CDN POPs; vector tiles as bandwidth optimization. Road graph at multiple LODs; offline pipeline ⇒ compressed binary adjacency in S3.

Adaptive rerouting: **store origin tile + super-tiles to destination; re-route only users whose final tile covers the incident tile.** Push: WebSocket** (mobile push

payloads too small). Kafka analytics ⇒ closed-road detection + ETA training. Web Mercator projection; geohash tile addressing; client-computable vs server-computed tile URLs.

18

DESIGN BLUEPRINT

Distributed Message Queue

Kafka/Pulsar-class event streaming: long retention, repeated consumption, per-partition ordering, configurable delivery. Batch size = throughput ↔ latency knob; 2-week retention. Producers/consumers → brokers (partitions) → data storage → state storage (offsets) → metadata storage → ZooKeeper (discovery + leader election). Message = (topic, partition, offset, timestamp, size, CRC, key, payload); immutable. Partition = WAL of segments; `hash(key) % partitions` routing ⇒ **ordering per partition**; partition key (e.g. `user_id`) pins ordering. Tricks: sequential-disk WAL + page cache; batching

everywhere; **pull consumers** (consumer controls rate; long-poll covers idle). Consumer groups: one consumer per partition; coordinator by hashed group name; rebalancing on join/leave/heartbeat-loss. Replication: leader + pull followers; **ack=all/1/0** = durability ↔ speed knob; ISR with lag thresholds. Delivery: at-most-once (async, commit-before-process); at-least-once (retry, commit-after-process; dupes on crash); exactly-once (costly: atomic offset+write or idempotent downstream). **Partitions only grow**; hot topic ⇒ more partitions. Deep dive: exactly-once atomic commit; delayed messages (temp topics + timing wheels); rebalancing storms.

19

DESIGN BLUEPRINT

Metrics Monitoring and Alerting

Internal metrics monitoring + alerting: operational metrics (not business), time-series storage, fast query, reliable alerting (email/phone/PagerDuty/webhook). 100M DAU $\Rightarrow$ 1,000 pools $\times$ 100 machines $\times$ 100 metrics = ~10M metrics. Retention: raw 7 days $\rightarrow$ 1-min rollups 30 days $\rightarrow$ 1-hour rollups 1 year; InfluxDB >250k writes/s; ~85% of queries hit last 26 h. Sources $\rightarrow$ collectors (**pull** scrapes `/metrics` vs **push** agents — big orgs run both; push gateway for short-lived jobs) $\rightarrow$ Kafka buffer $\rightarrow$ stream processing (Flink/Spark) $\rightarrow$ TSDB $\rightarrow$ query service $\rightarrow$ Grafana + alerting (rule cache $\rightarrow$ alert manager $\rightarrow$ store $\rightarrow$ Kafka $\rightarrow$

channel consumers). Data: metric name + labels + (timestamp, value); indexed by (name, labels) — **keep label cardinality low**; delta timestamp encoding; downsampling; cold storage. Alerts in YAML (`expr: up == 0, for: 5m`); dedup/merge/filter; at-least-once delivery. Kafka before TSDB $\Rightarrow$ no loss on TSDB outage; partition by metric name; consistent-hash collector assignment avoids double-scraping. Aggregation placement: agent (simple) / ingestion (less volume, loses precision) / query side (no loss, slower queries).

20

DESIGN BLUEPRINT

Ad Click Event Aggregation

Clicks at Facebook/Google scale for billing/RTB: per-ad counts over windows, top-100/min, attribute filters; money-grade correctness, late/duplicate events, partial-failure resilience. 1B clicks/day $\Rightarrow$ ~10k avg QPS, 50k peak; 2M ads; 0.1 KB/click $\Rightarrow$ 100 GB/day; e2e latency a few minutes. `GET /v1/ads/{id}/aggregated_count` (filter IDs, e.g. 001 = non-US); `GET /v1/ads/popular_ads?count=N&window=M&filter=...`. Kafka (raw) $\rightarrow$ aggregation (MapReduce DAG: map $\rightarrow$ aggregate $\rightarrow$ reduce) $\rightarrow$ Kafka (per-minute aggregates + top-N; **enables end-to-end exactly-once via atomic commit**) $\rightarrow$ Cassandra $\rightarrow$ query API. **Recalculation service** replays raw data on bugs on a

dedicated pipeline; raw kept (Cassandra or S3+Parquet cold) for backfill. Data: raw (ad_id, click_timestamp, user, ip, country); aggregated (ad_id, click_minute, filter_id, count) with star-schema filter dimensions (fast, combinatorial explosion). **Kappa** over Lambda (one streaming path). Tumbling windows for counts; sliding for top-N; per-node heap. **Event time (not processing time)** + watermark for late events (longer = fewer misses, more latency); end-of-day reconciliation. Exactly-once: atomic offset+downstream commit or idempotent sinks. **Hotspot ads**: extra nodes, split group, merge back. Per-minute state snapshots + Kafka offsets; nightly reconciliation batch compares recomputed vs stored.

21

DESIGN BLUEPRINT

Hotel Reservation System

Chain hotel reservations: browse, reserve with full upfront payment, cancel, 10% overbooking, daily prices. 5,000 hotels, 1M rooms, 70% occupancy $\Rightarrow$ ~240k reservations/day $\approx$ **3 TPS** (funnel: 3 bookings $\leftarrow$ 30 reservation pages $\leftarrow$ 300 detail views). 73M `room_type_inventory` rows fits one DB; booking.com scale $\Rightarrow$ 16 shards $\approx$ 1,875 QPS/shard. `POST /v1/reservations` includes **reservationID = idempotency key**. Microservices: CDN $\rightarrow$ API gateway (rate limit/auth) $\rightarrow$ hotel service (cached static data), rate service (occupancy-based pricing), **reservation + inventory in ONE service/DB (keeps ACID)**, payment, admin. Cross-service $\Rightarrow$ 2PC (blocking) or Saga (eventual). Relational

DB (read-heavy, low-write; ACID prevents double-charge). **room_type_inventory** (hotel_id, room_type_id, date, total_inventory, total_reserved), pre-populated by daily cron; availability: `reserved + requested  $\leq$  110%  $\times$  inventory`. Double-click $\Rightarrow$ idempotency key as unique constraint (button disable insufficient). Last-room contention: **pessimistic locking NOT recommended** (deadlocks, unscalable) $\Rightarrow$ **optimistic version column** (low contention) or CHECK constraints. Redis `hotelID_roomTypeID_{date}` $\rightarrow$ `available` with TTL, CDC-synced; transient inconsistency OK — **DB is the guardrail**. Shard by `hash(hotel_id)`; cold-archive old reservations.

22

DESIGN BLUEPRINT

Distributed Email Service

Gmail-scale: send/receive with attachments, folders, filters, full-text search, anti-spam; HTTP-based, SMTP/IMAP/POP compatible. 1B users; 10 sent/day $\Rightarrow$ 100k/s; 40 received/day $\times$ 50 KB $\Rightarrow$ ~730 PB/yr metadata; 20% attachments $\times$ 500 KB $\Rightarrow$ ~1,460 PB/yr; 25 MB cap. [POST /v1/messages](#) ; paginated folder reads; WebSocket push (long-poll fallback); RFC 6154 folders. **Send:** queue (attachments as S3 references) $\rightarrow$ SMTP workers (spam/virus checks, exponential-backoff retry) $\rightarrow$ destination MX. **Receive:** SMTP servers $\rightarrow$ workers $\rightarrow$ metadata DB + Redis (recent mail) + object store + search index + realtime servers. Data: emails partitioned

by user_id, timeuuid clustering $\Rightarrow$ time-sorted; **denormalize read/unread tables** (Cassandra can't filter non-key columns); threads via Message-Id/In-Reply-To/References; attachments table (dedup across users). **Consistency over availability (CAP)** — data loss unacceptable, writes may briefly block on failover. Search: Elasticsearch per user_id, async via Kafka (operational burden) vs custom LSM-tree engine (write-optimized, heavy engineering). Deep dive: deliverability (dedicated IPs, transactional/marketing separation, 2–6 week IP warmup, spammer bans, feedback loops, SPF/DKIM).

23

DESIGN BLUEPRINT

S3-like Object Storage

S3-class: bucket CRUD, object upload/download/listing, versioning; tiny + huge objects; petabyte scale; 99.9999% durability, 99.99% availability; immutable objects; write-once/read-many (95% reads). 100 PB/yr; mix 20% <1 MB / 60% 1–64 MB / 20% >64 MB ⇒ ~0.68B objects, ~0.68 TB metadata. `PUT/GET /{bucket}/{object}`; "folders" via prefix; **multipart upload** (initiate → upload parts → complete). LB → stateless API service (IAM + metadata + data orchestration) → IAM → data store (objects by UUID) + metadata store; placement service (virtual cluster map; 5–7 node Paxos/Raft) → data nodes. **Metadata/data separation (UNIX inodes)**. Data: per-node `object_id → (filename, offset, size)`

(SQLite/RocksDB). Metadata: buckets table (small, single server + replicas); objects sharded by `hash(bucket_name, object_name)` — **bucket_id alone hotspots**. Versioning: TIMEUUID column; new version ⇒ new `object_id`; deletes = tombstone → 404. **Replication vs EC**: 3 replicas across failure domains ≈ 6 nines; **replicate-before-ack = strong consistency**. EC 8+4 ≈ 50% overhead vs 200%, 11 nines, but slower reads + harder to build. **Small-file fix**: WAL-pack into few-GB files; pin files to cores; per-file and per-object checksums. Sharded listing is painful ⇒ accept slow listing or denormalized per-bucket listing table. GC via compaction (copy live, transactional remap).

24

DESIGN BLUEPRINT

Real-time Gaming Leaderboard

Monthly-tournament board (win = 1 point): top 10, any user's rank, ± 4 neighbors; real-time. 5M DAU; 10 matches/player/day $\Rightarrow$ ~ 500 avg / 2,500 peak write QPS; 25M users $\times 26 \text{ B} \approx 650 \text{ MB} \Rightarrow$ one Redis. `POST`

`/v1/scores` (**game servers only, never clients** — anti-tamper); `GET /v1/scores`; `GET /v1/scores/{user}`. **Redis sorted set per month** — `ZADD/ZINCRBY` $O(\log N)$; `ZREVRANGE 0 9` = top 10; `ZREVRANK` = user's rank $\rightarrow$ rank ± 4 = neighbors. Relational rank queries die on full sorts; B-

tree index+LIMIT only gives top-10. MySQL user profiles + match-history = **disaster-recovery source of truth** (WAL replay). Scaling: **range-partition by score** (top-10 = highest shard; rank = intra-shard + $O(1)$ higher-shard counts) beats hash-partitioning (scatter-gather, no easy rank). Cassandra/Dynamo by month $\Rightarrow$ hot partition; `user_id % N` write-sharding $\Rightarrow$ read scatter-gather; rank fallback = percentiles via cron. 2 $\times$ memory on write-heavy nodes (snapshots); ties share rank, break by last-played.

25

DESIGN BLUEPRINT

Payment System

E-commerce pay-in (buyer → merchant via PSP) + pay-out (to sellers), reconciled externally. Correctness >> throughput: 1M/day ⇒ ~10 TPS; single currency; no card data stored (PCI via PSP-hosted pages).

`POST /v1/payments` — **amounts as strings, never doubles**; each order carries

`payment_order_id` = **globally-unique**

idempotency key forwarded to the PSP.

Payment service (risk/AML) → executor → PSP → card schemes; **double-entry ledger** (every movement = debit + credit; all entries sum to zero); wallet (merchant balances); hosted-payment-page flow (token → redirect → async webhook); retry queue + dead-letter queue; nightly reconciliation vs

settlement files. Data (SQL — ACID): `payment_events`; `payment_orders` (status machine +

`ledger_updated` / `wallet_updated` flags + sweeper for stuck payments). **Exactly-once = at-least-once + at-most-once:**

exponential-backoff retries (`Retry-After`) + idempotency (unique DB constraint; PSP nonce dedup). **Two double-charge causes:** double-click; PSP success + downstream failure before ack. Replication lag ⇒ primary reads or consensus DBs. Async pub/sub for side-effects (ledger, wallet, notifications). Delayed payments (risk review, 3D Secure); pending + webhook.

26

DESIGN BLUEPRINT

Digital Wallet

Wallet transfers at **1M TPS** (2M legs); 99.99% availability; transactional; replayable from genesis. RDBMS node $\approx$ 1,000 TPS $\Rightarrow$ 2,000 nodes naively — goal: raise per-node TPS. **POST** `/v1/wallet/balance_transfer` (from, to, amount as **string**, currency, **transaction_id** = **idempotency key**). Stateless service + sharded in-memory `map<user_id, balance>` (fast, not cross-shard atomic) $\Rightarrow$ distributed transactions: 2PC (correct; lock contention + coordinator SPOF — rejected at scale), **Try-Confirm/Cancel (preferred)**, Saga. **TC/C**: phase 1 **deducts from sender first (never credit first)** — users can't spend mid-flight; phase 2 confirms (credit) or cancels (compensate); two independent

transactions, parallelizable, DB-agnostic.

Saga: linear local transactions + compensating rollbacks; simpler, no parallelism. Out-of-order (cancel-before-try) $\Rightarrow$ flags in phase-status tables; crash recovery from same tables. **Event sourcing**: commands (non-deterministic, FIFO) $\rightarrow$ events (immutable facts) $\rightarrow$ **deterministic state machine** (no I/O, no randomness) $\rightarrow$ state. Answers "balance at time T?", validates balances by recompute, dual-runs new code. **Only the event log needs Raft** — state/snapshots regenerable; mmap'd append-only disk (page cache) over Kafka; RocksDB (LSM) for state; HDFS snapshots bound replay. CQRS read-only projections (poll $\rightarrow$ push for real-time UX).

27

DESIGN BLUEPRINT

Stock Exchange

Stocks, limit orders, place/cancel, normal hours; real-time trades + order book (L1/L2/L3); risk checks + fund withholding. ~100 symbols; 1B orders/day over 6.5 h ⇒ **~43k avg QPS, ~215k peak (5×, heaviest at open)**; 99.99% (8.64 s/day); ms round-trip, **p99-focused (GC pauses called out)**.

`POST /v1/order` (symbol, side, **price/quantity as long**); market-data and candle endpoints. **Trading path**: client → broker → client gateway (validate/auth/rate-limit, lightweight) → order manager (risk, funds) → **sequencer** (sequence IDs: fairness, replay, exactly-once) → **matching engine** (FIFO per price level, emits fills) → executions back.

Market data: fills → publisher (rebuilds book, candles) → in-memory columnar store (KDB) for realtime; historical DB after close. **Reporter**: orders + executions → DB for history/tax/compliance/settlement.

Order book: PriceLevel(price, volume, doubly-linked orders) + limitMap + orderMap ⇒ place/match/cancel all O(1); bestBid/bestOffer cached. Candles in preallocated **ring buffers** (lock-free, cache-line padded). **Ultra-low-latency**: real exchanges run on **one giant server** — strip critical path (even logging), **mmap'd event bus** (`/dev/shm` , no disk), one CPU-pinned loop (no context switches/locks) ⇒ tens of μs vs tens of ms. Event sourcing: immutable transitions on the bus; sequencer as single-writer. **HA**: hot/warm standby, heartbeats, Raft leader election, reliable-UDP replication, frequent backups, chaos engineering. Market-data fairness: multicast over reliable UDP. **Kafka rejected as sequencer** (latency). FIX protocol; colocation as VIP service; DDoS-hardened public market-data endpoints.

28

DECISION INDEX

Cross-cutting patterns 1/3

Start from the requirement, then choose the pattern. Every pattern buys one property by accepting another cost.

Pattern	When to use	Notes	Chapters
Consistent hashing	Nodes join/leave; keys must not all move (caches, KV, crawl servers, collectors)	Ring; clockwise ownership; virtual nodes for balance (weighted for heterogeneous capacity); only K/n keys move	1, 5, 6, 9, 11, 17, 20
Rate limiting	Throttle API/client traffic, abuse control	Token bucket (bursty, default, Redis-friendly); leaky bucket (smooth outflow); sliding-window counter (practical Redis compromise); fixed-window has 2× edge-burst trap; sliding-window log is memory-heavy	4, 8, 10, 28
Fanout write vs read	Distribute content to followers	Write (push): fast reads, costly for celebrities. Read (pull): cheap writes, slow reads. Hybrid : push normal, pull celebrities	11, 12, 17
CQRS	Reads/writes have different scale or shape	Separate read models from write path; event-sourced state + read-only projections	20, 27
Sharding	DB/KV outgrows one box	Hash a key every query filters on (<code>hotel_id</code> , <code>user_id</code>); range for ordered access (leaderboard scores); watch hot keys / celebrity shards (dedicated shard); denormalize to avoid cross-shard joins	1, 6, 11, 13, 22, 23, 24, 25

DECISION INDEX

Cross-cutting patterns 2/3

Pattern	When to use	Notes	Chapters
Caching	Hot reads, rarely-written data	Cache-aside + LRU; cache IDs not full objects; geohash keys for geo (not raw GPS); precomputed top-k per trie node	1, 8, 11, 13, 16, 22, 25
Message queues	Decouple, buffer, smooth bursts	Topics per concern; pull consumers + consumer groups with rebalancing; delay via temp topics + timing wheels	10, 11, 12, 19, 20, 21
CDNs	Static/heavy content at edge	Video (DASH/HLS); map tiles; cache only popular videos; CDN as upload hub	1, 14, 18, 22
Geo-indexing	Proximity / map problems	Geohash (easy; always query 8 neighbors); quadtree (density-adaptive, k-nearest); S2/Hilbert (geofencing, hardest)	16, 17, 18

DECISION INDEX

Cross-cutting patterns 3/3

CAP/PACELC postures: CP — email metadata (23), object-store write path / replicate-before-ack (24), hotel DB (22), payment SQL (26). AP/eventual — KV AP mode (6), nearby friends (17), maps location (18), news feed (11). Money ⇒ **idempotency + reconciliation + ACID islands** (Saga/TC-C, nightly settlement: 21, 22, 26, 27).

Pattern	When to use	Notes	Chapters
LSM vs B-tree	Write-heavy storage	LSM (memtable → SSTable): Cassandra, RocksDB; Bloom filters skip SSTables; B-tree for reads/ordered scans	6, 23, 24, 27
Idempotency	Any money/booking flow	Key ⇒ unique DB constraint; forward key to PSP; reservationID, payment_order_id, transaction_id	22, 26, 27
Exactly-once	Correctness-critical pipelines	At-least-once retries + idempotent sinks; atomic offset+downstream commit; double-entry ledger (sums to zero)	19, 21, 26, 27
Event sourcing	Auditability, replay, debugging	Immutable event log = source of truth; deterministic state machine; snapshots bound replay	27, 28

REVIEW

Common pitfalls and wrap-up

Use this page before an interview hand-off or architecture review. Each point names a common failure mode or a stronger closing move.

Pitfalls: (1) Jumping to solutions before clarifying scope. (2) Fixed-window limiter $\Rightarrow$ 2 $\times$ edge bursts (Ch 4). (3) Predictable short URLs from sequential IDs (Ch 8). (4) 301 caches in browser $\Rightarrow$ use 302 for click analytics (Ch 8). (5) Polling/long-polling for chat $\Rightarrow$ WebSocket (Ch 12). (6) Global chat sequence IDs are overkill — per-channel suffices (Ch 12). (7) Presence fanout only for small groups (Ch 12). (8) Real-time trie updates at billions/day infeasible — batch (Ch 13). (9) Geohash boundary — query neighbors (Ch 16). (10) Resharding without consistent hashing (Ch 5). (11) Single notification server / DB inside it — SPOF (Ch 10). (12) Pessimistic locking $\Rightarrow$ optimistic version column (Ch 22). (13) Event time vs processing time (Ch 21). (14) Never trust the client for scores/payments (Ch 25). (15) Amounts as doubles $\Rightarrow$ strings/ longs (Ch 26, 28). (16) Replicate only the event log — state regenerates (Ch 27). (17) GC pauses kill p99 (Ch 28). (18) Sharding object metadata by bucket_id alone $\Rightarrow$ hotspots (Ch 24).

Wrap-up (framework step 4): recap chosen trade-offs; walk 1M $\rightarrow$ 10M scaling (what breaks first — usually single master, fanout path, or hot shard); per-component error handling (retry, dead-letter, failover); monitoring (queue depth, cache hit rate, p99); one follow-up improvement; ask what they'd like deeper.

REFERENCE

Sources and scope

This handbook is a condensed working reference. Use it to select trade-offs and prepare a design discussion; return to the full chapter notes and diagrams when implementation details matter.

Primary source

liquidslr/system-design-notes. Third-party notes covering *System Design Interview – An Insider's Guide*, Volumes 1 and 2. <https://github.com/liquidslr/system-design-notes>

Scope

- All 28 chapter summaries from the authoritative distilled reference are included.
- The chapter manifest, interview framework, estimation sheet, pattern index, pitfalls, and wrap-up prompts are preserved.
- Product numbers are estimation examples from the source notes, not production guarantees.
- Architecture choices still require current product requirements, measurements, security review, and operational testing.

Recommended use

For each project: define scope and service-level objectives; estimate load; sketch the read and write paths; select the critical consistency model; plan failure recovery; then verify what breaks at 10× scale.